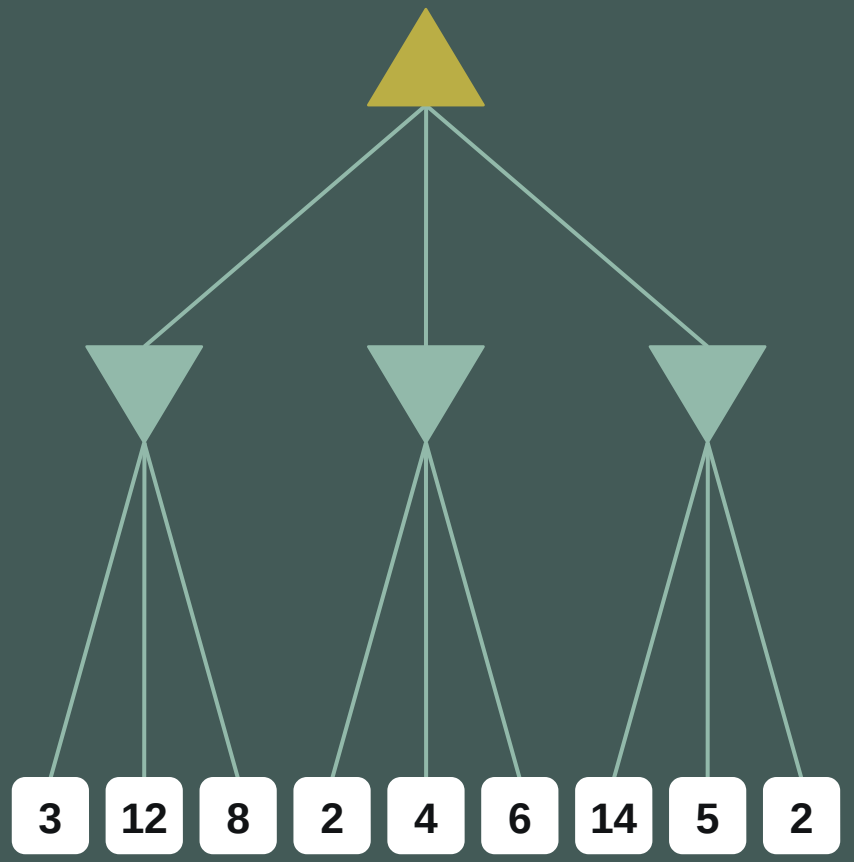
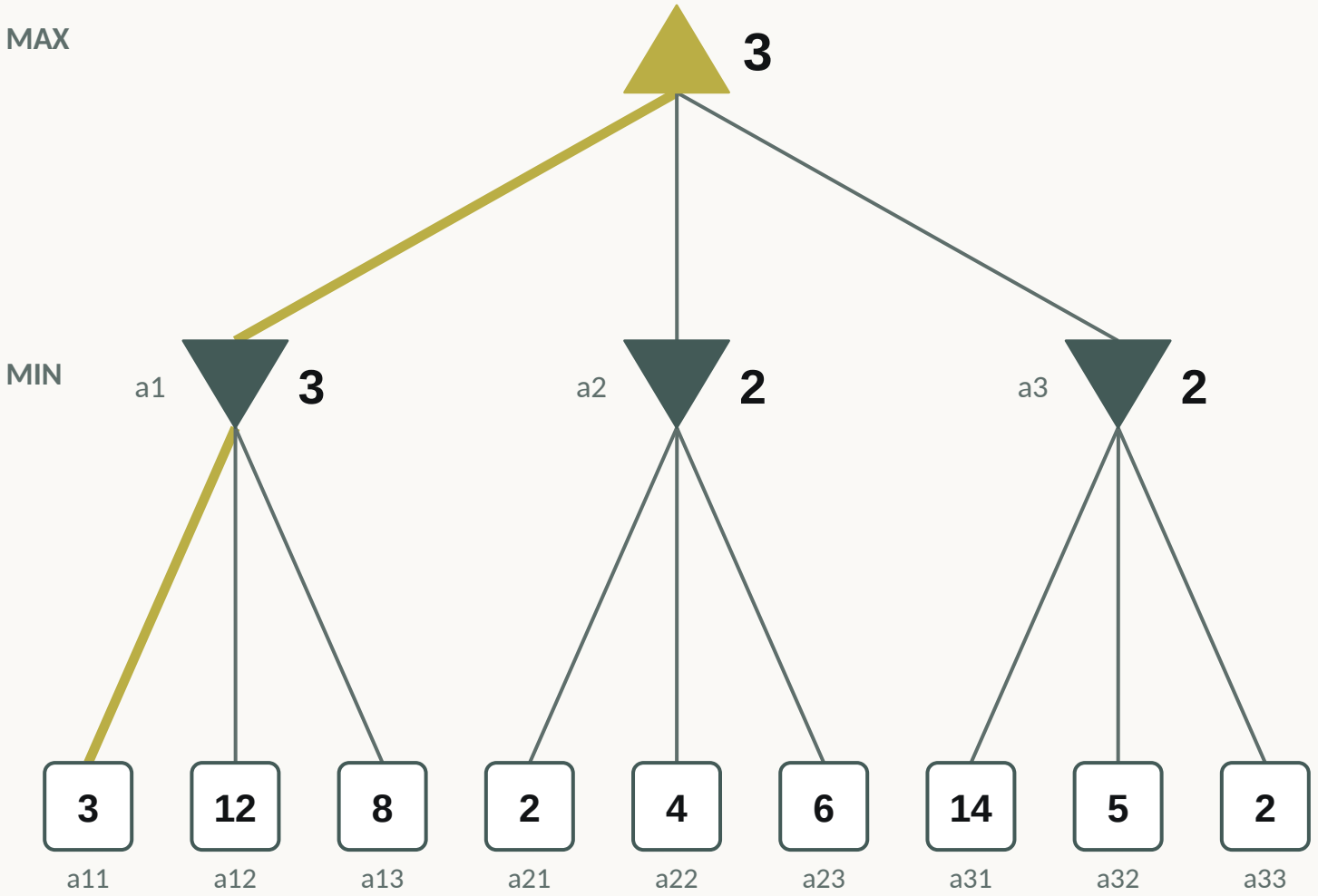
Adversarial search

Minimax and Alpha-Beta in Prolog

From the game tree to a running program



One tree, two algorithms



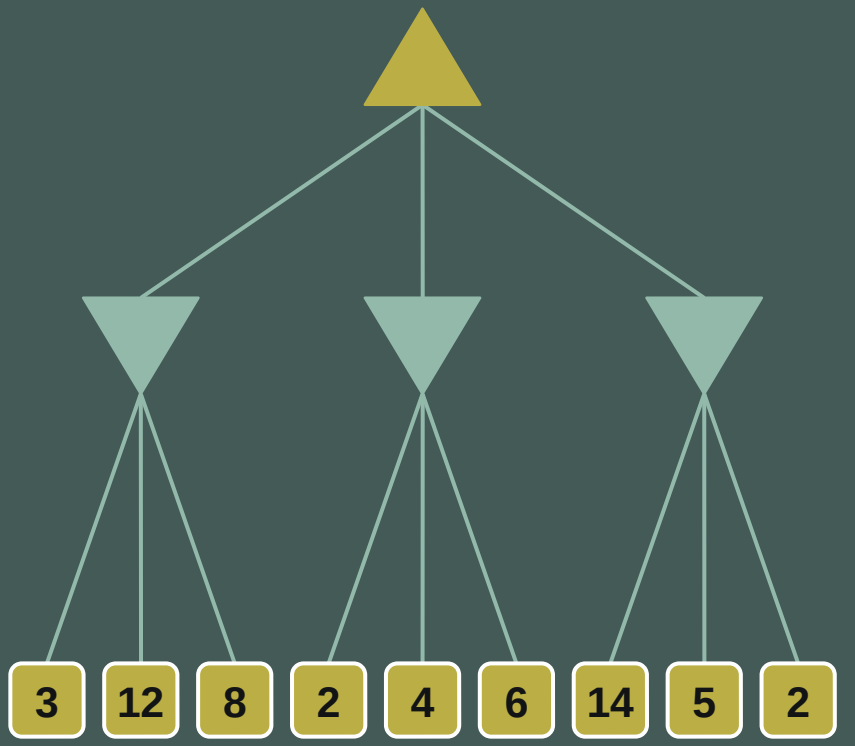
3

is what MAX can guarantee by playing a1: MIN will answer with a11.

We will write this tree as Prolog facts and solve it twice: first with minimax, which looks at every leaf, then with alpha-beta, which skips the leaves that cannot change the decision.

Part 1: Minimax

Explore every leaf and back the values up the tree



The tree as Prolog facts

```
% The game tree
move(root, a1).  move(root, a2).  move(root, a3).
move(a1, a11).   move(a1, a12).   move(a1, a13).
move(a2, a21).   move(a2, a22).   move(a2, a23).
move(a3, a31).   move(a3, a32).   move(a3, a33).

% Utility of the leaves
utility(a11, 3).  utility(a12, 12). utility(a13, 8).
utility(a21, 2).  utility(a22, 4).  utility(a23, 6).
utility(a31, 14). utility(a32, 5).  utility(a33, 2).

% Who moves
max_to_move(root).
min_to_move(a1).  min_to_move(a2).  min_to_move(a3).
```

1

move/2: the edges

Each fact is one legal move. findall over move/2 gives the children of a node.

2

utility/2: the leaves

Only terminal positions have a utility. Here, gains for MAX.

3

Whose turn it is

max_to_move/1 and min_to_move/1 tell best/3 whether to maximize or minimize.

minimax/3 in four steps

```
minimax(Pos, BestNextPos, Val) :-
1       findall(NextPos, move(Pos, NextPos), NextPosList),
        (   NextPosList \= []
2       ->  findall(Value-ChildPos,
                    ( member(ChildPos, NextPosList),
                      minimax(ChildPos, _, Value) ),
                    ValsPosList),
3           best(Pos, ValsPosList, Val-BestNextPos)
4       ;   leaf_value(Pos, Val),
            BestNextPos = none
        ).
```

leaf_value/2 is utility/2 plus a print, so we can watch which leaves are visited.

1 Collect the children

findall builds the list of positions reachable with one move.

2 Evaluate each child

minimax calls itself on every child and keeps Value-ChildPos pairs.

3 Pick the best pair

best/3 maximizes or minimizes depending on who moves.

4 Or stop at a leaf

With no children, the value is the utility and there is no next move.

best/3: MAX takes the largest, MIN the smallest

```
best(Pos, ValsPosList, Best) :-
    (   max_to_move(Pos) -> max_member(Best, ValsPosList)
    ;   min_to_move(Pos) -> min_member(Best, ValsPosList)
    ).
```

At root, after evaluating the three MIN nodes:

3-a1 | **2-a2** | **2-a3** → `Val = 3` `BestNextPos = a1`

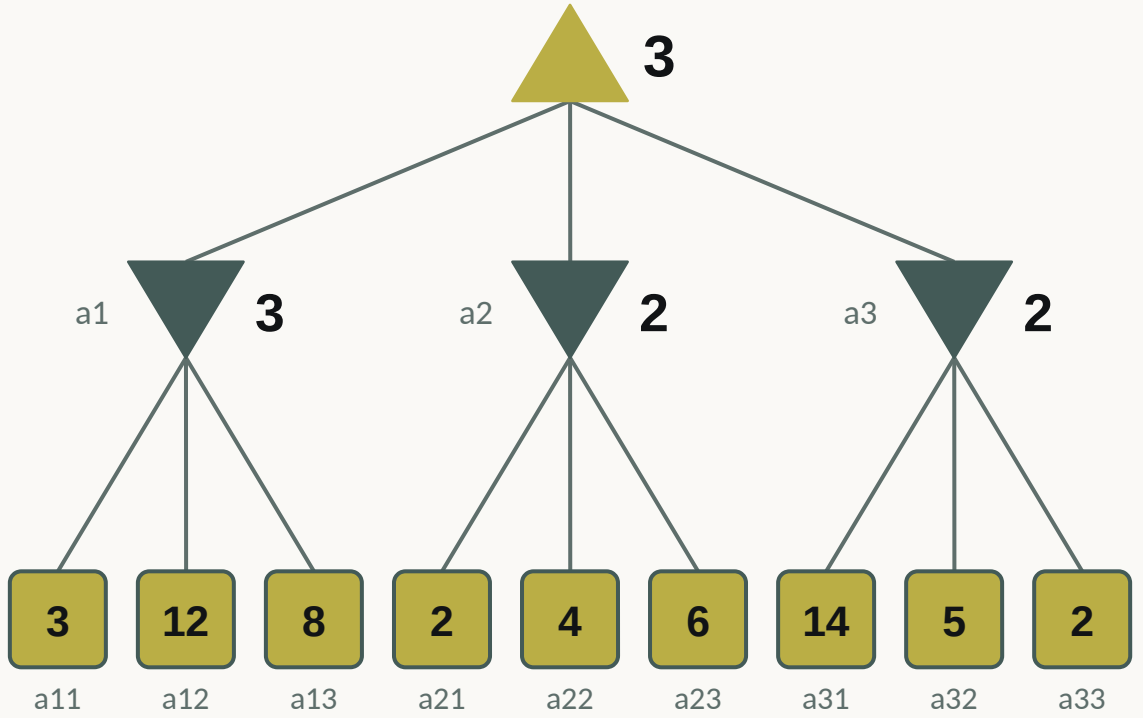
max_member/2 compares pairs in Prolog's standard order: first the value, then the position. Writing the value first is what makes this one-liner work.

Why pairs?

Keeping the position next to its value means that when we find the best value, we already know which move produced it. No second search is needed.

Running minimax

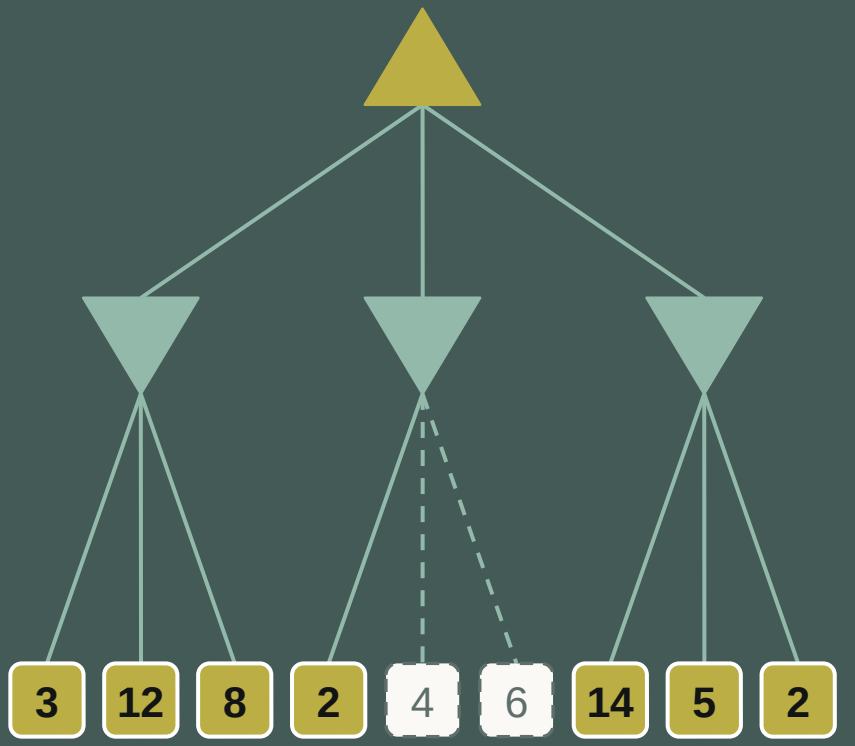
```
?- minimax(root, Move, Val).
  leaf a11 = 3
  leaf a12 = 12
  leaf a13 = 8
  leaf a21 = 2
  leaf a22 = 4
  leaf a23 = 6
  leaf a31 = 14
  leaf a32 = 5
  leaf a33 = 2
Best move: a1   Value: 3
```



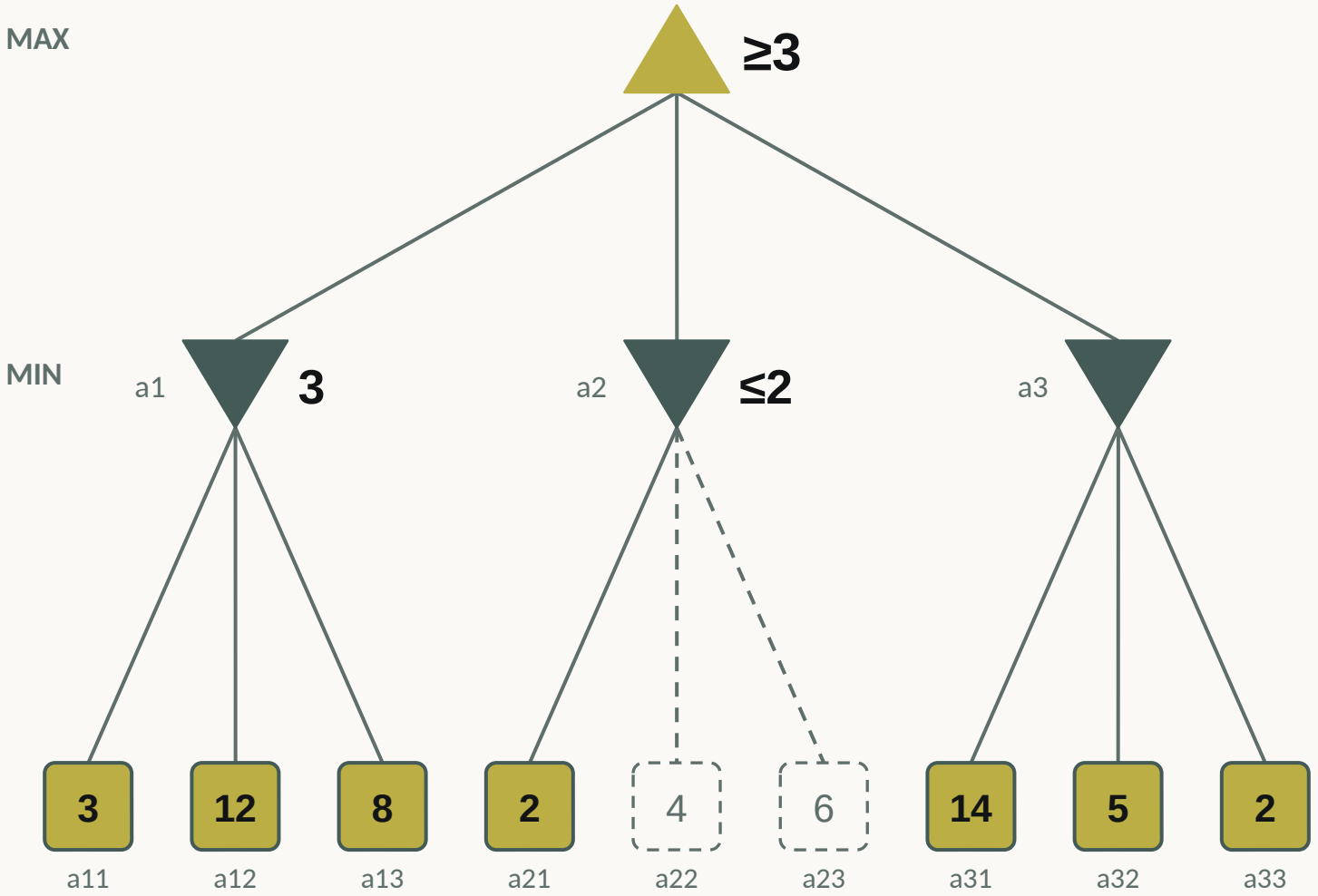
9 of 9 leaves evaluated. Minimax never skips anything.

Part 2: Alpha-beta

Same decision, fewer leaves



Stop when the answer can no longer change



1 **α: MAX's safe value**

The best MAX can already guarantee. After a1, α = 3.

2 **β: MIN's safe value**

The best MIN can already guarantee in the current branch.

3 **The cut at a2**

a21 = 2 means MIN can force ≤ 2 here. MAX already has 3, so a22 and a23 can't matter.

From pseudocode to Prolog

Pseudocode (slide 27)

```
function MAX-VALUE(state, α, β)
  if TERMINAL-TEST(state)
    then return UTILITY(state)
1   v ← −∞
2   for a, s in SUCCESSORS(state) do
3     v ← MAX(v, MIN-VALUE(s, α, β))
4     if v ≥ β then return v
5     α ← MAX(α, v)
  return v
```

Prolog

```
max_value([], _, _, Best, V, Best, V).
2 max_value([C|Cs], Alpha, Beta, Best0, V0, Best, V) :-
3     alphabeta(C, Alpha, Beta, _, CV),
      (   CV > V0 -> V1 = CV, Best1 = C
      ;   V1 = V0,  Best1 = Best0
      ),
4     (   V1 >= Beta                          % beta cut
      ->  prune(Cs), Best = Best1, V = V1
5     ;   Alpha1 is max(Alpha, V1),
          max_value(Cs, Alpha1, Beta, Best1, V1, Best, V)
      ).
```

Same numbers, same idea. Badge 1 (v ← −∞) has no line in max_value: it is the -1000 passed in by alphabeta/5. A loop becomes recursion over the list [C|Cs], and the base case max_value([], …) is the final “return v”.

alphabeta/5: deciding who moves

```
alphabeta_search(Pos, Best, Val) :-
1       alphabeta(Pos, -1000, 1000, Best, Val).

alphabeta(Pos, Alpha, Beta, Best, Val) :-
2       findall(C, move(Pos, C), Children),
3       (   Children = []                          % terminal
        ->  leaf_value(Pos, Val), Best = none
4       ;   max_to_move(Pos)
        ->  max_value(Children, Alpha, Beta,
                      none, -1000, Best, Val)
5       ;   min_value(Children, Alpha, Beta,
                      none, 1000, Best, Val)
        ).
```

1. **Widest window first**

   −1000 and 1000 stand in for −∞ and +∞.

2. **Collect the children**

   Same findall as in minimax.

3. **Leaf: return its utility**

   This is TERMINAL-TEST.

4. **MAX node**

   max_value starts with V = −∞.

5. **MIN node**

   min_value starts with V = +∞.

min_value/7 is the mirror image

```
min_value([], _, _, Best, V, Best, V).
min_value([C|Cs], Alpha, Beta, Best0, V0, Best, V) :-
    alphabeta(C, Alpha, Beta, _, CV),
1   (   CV < V0 -> V1 = CV, Best1 = C
    ;   V1 = V0,  Best1 = Best0
    ),
2   (   V1 =< Alpha                          % alpha cut
    ->  prune(Cs), Best = Best1, V = V1
3   ;   Beta1 is min(Beta, V1),
        min_value(Cs, Alpha, Beta1, Best1, V1, Best, V)
    ).

prune([]).
prune(Cs) :- Cs \= [],
    format("  prune: skip ~w~n", [Cs]).
```

1 Keep the smallest value

CV < V0 instead of CV > V0.

2 Alpha cut

If V ≤ α, MAX already has something better elsewhere: stop.

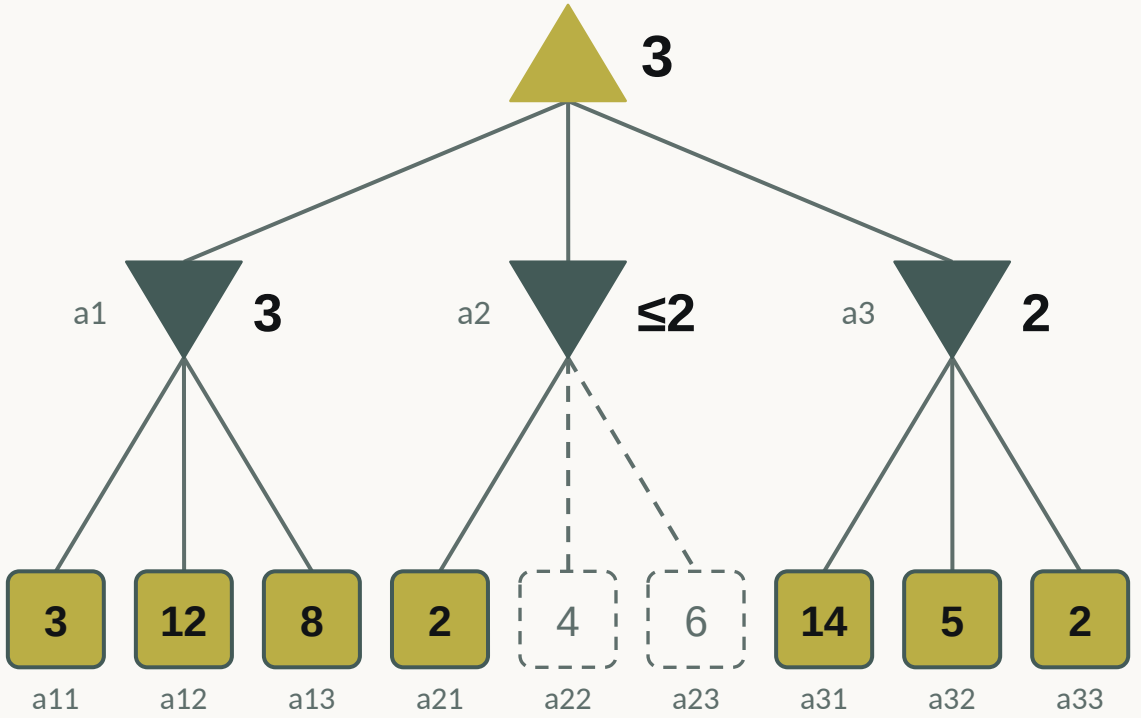
3 Tighten β

β only goes down, and it is passed to the remaining siblings.

prune/1 just prints the siblings that were skipped.

Running alpha-beta

```
?- alphabeta_search(root, Move, Val).
  leaf a11 = 3
  leaf a12 = 12
  leaf a13 = 8
  leaf a21 = 2
  prune: skip [a22,a23]
  leaf a31 = 14
  leaf a32 = 5
  leaf a33 = 2
Best move: a1   Value: 3
```



7 of 9 leaves evaluated, and exactly the same answer: a1 with value 3.

Predict first, then run

1

Change one leaf

Set utility(a21, 4). Which leaves are pruned now? Does MAX still choose a1?

2

Reorder the moves

List move(a3, a33) first. How many leaves are evaluated this time?

3

Add a depth limit

Pass a Depth argument and, at 0, call an eval/2 instead of utility/2.

To run it

```
$ swipl minimax_alphabeta.pl
?- minimax(root, Move, Val).
?- alphabeta_search(root, Move, Val).
```

Same answer, less work

Alpha-beta never changes the answer of minimax.
Good move ordering makes it prune even more.

9 leaves with minimax

7 with alpha-beta

5 with a33 explored first